Frontend components

Shared building blocks, grouped by folder. All are function components styled with Tailwind; props are listed as they exist in the code. The `token` prop appears on several components because they call the API themselves (through `@/api/*`) rather than receiving data from a page.

The PDF overlay engine

`PdfOverlayViewer` — `components/case/PdfOverlayViewer.tsx`

Renders **one page at a time** of a case document with bounding boxes drawn **live on top**. Nothing annotated is ever generated or stored; the boxes vanish the moment `overlays` is empty. Wrapped in an error boundary (react-pdf can throw synchronously on an expired or blocked URL, which would otherwise unmount the whole case page). Page width is measured from the card and capped at 560 px so it never spills into the adjacent column. Uses `react-pdf` (pdf.js) with a Vite-bundled worker.

Prop	Type	Meaning
`fileUrl`	`string`	A signed SAS URL for the PDF (from `CaseDocumentSummary.file_url`)
`overlays`	`OverlayBox[]`	Boxes to draw

```
interface OverlayBox { box: BoundingBox; color: OverlayColor; label: string }
type OverlayColor = "destructive" | "warning" | "ai" | "field"
// BoundingBox = { page (1-based), x, y, width, height }  — 0-1 page fractions, origin top-left
```

The four colours are **fixed per source**, so a reviewer learns the legend once:

Colour	Style	Source
`destructive`	solid red	ELA recompression region
`warning`	solid orange	Copy-move cluster
`ai`	**dashed** blue, "approximate"	Vision-model judgment (visual review, signature comparison) — styled to look less precise
`field`	solid purple	Rule-based field exception (failed field validation, cross-document mismatch)

Because boxes are page fractions, they are resolution-independent and land correctly regardless of the rendered size.
Used in: `CaseDetailPage` (middle column).

`getCheckOverlays(checks, crossDocumentFindings?, documentId?) → OverlayBox[]` — `DocumentChecksPanel.tsx`

The **single function that turns stored check rows into boxes**, shared by the viewer and the findings list so the highlighted regions and the listed findings can never disagree (both read the same `document_checks` rows). It reads ELA/copy-move findings' `bounding_box`, flagged field-validation sub-checks' `regions`, visual-review boxes, and cross-document regions (only those belonging to `documentId`, so each document shows its own side of a mismatch).

Case components — `components/case/`

Component	Props	Purpose / where used
`DocumentChecksPanel`	`checks: DocumentCheck[]`, `crossDocumentFindings: CrossDocumentFinding[]`, `hasEnoughDocumentsForCrossCheck: boolean`, `signatureMatches?: SignatureMatch[]`	One expandable card per check with its findings, a synthetic cross-document card, and the signature-comparison card (badge colours: consistent green, possibly-consistent/identical-reuse amber, inconsistent/different-signer red, cannot-determine grey). Handles the differing `details` shapes (list of findings vs. dict of sub-checks vs. flat issuer dict). Duplicate findings link to the matched case. *CaseDetailPage.*
`CaseDecisionPanel`	`caseDetail: CaseDetail`, `role: UserRole | undefined`, `token: string`	Approve / reject / escalate with confirmation dialogs. Approve is disabled until `pipeline.complete` and an assessment exists; above low risk it requires a ≥ 10-character justification. **These are conveniences — every rule is enforced by the API.** Invalidates the `case`, `caseAuditLog` and `cases` queries on success. Escalate requires a reason and moves the case to L2. Non-reviewers get a read-only "Review status" card with the decision history instead; a Reviewer L1 on an L2 case (`caseDetail.can_act` false) gets a read-only "Escalated · L2" card. The decision history shows each actor's role label (`actor_role`). *CaseDetailPage.*
`CaseReportExport`	`caseId: string`, `token: string`	"Export report" button (`POST /cases/{id}/reports`; opens the new PDF in a tab, with an "Open PDF" link as a popup-blocker fallback) and a collapsible history of previous reports. Reviewers only (hidden in a platform admin's support view). *CaseDetailPage.*
`ActivityTimeline`	`entries: AuditLogEntry[]`, `isLoading: boolean`	The case's audit trail, **newest first** (the API returns oldest first). Sentences come from `lib/audit.ts` `describeAuditEvent`. *CaseDetailPage.*
`CaseBadges`	exports `CaseFlagBadge({flag})`, `TierBadge({tier})`, `RiskBadge`, `riskStripeClass(tier)`	The coloured risk-tier pill and "Escalated · L2" marker. *Queue, Dashboard, MyCases, Detail.*

Upload components — `components/upload/`

Component	Props	Purpose
`FileDropzone`	`files: FileWithProgress[]`, `onFilesAdded(files: File[])`, `onFileRemoved(index: number)`, `disabled?`	Drag-and-drop / click-to-browse list with per-file progress and error. `FileWithProgress = { file: File; progress?: number (0-100); error?: string }`. The picker only offers PDFs (`accept` from `lib/uploadLimits.ts`) and the zone states the rules ("PDF only · up to 10.0 MB each · not password-protected"); the page refuses non-PDF, empty and over-limit files before upload, and shows the server's specific rejection message per file. *NewCasePage.*

Component	Props	Purpose
`SignatureReferenceCreator`	`caseId`, `documentId`, `documentFilename`, `token`, `onCreated(ref: SignatureReference)`, `onClose()`	Modal that fetches a signed URL, renders the PDF with `react-pdf`, lets the reviewer **drag a box over a signature** (pointer coordinates and the box are both relative to the rendered page wrapper, so they always agree), takes a required `person_name`, then `POST`s the reference. Error-boundary wrapped. Available to whoever uploaded (the case owner) and to the company's reviewers. *NewCasePage*.

Settings components — `components/settings/`

Component	Props	Purpose
`SettingsShell`	`active: SettingsTab` (`"issuers" | "risk-rules" | "users"`), `title?`, `description`, `actions?`, `children`; also exports `StatusLine({error, ok})`	Shared page frame and tab bar for the company settings screens (Issuer Registry, Risk Rules); shows the company picker to platform admins ("Changes apply to this company only and are recorded in its audit log.")
`Modal`	`title`, `description?`, `children`, `onClose`, `busy?`, `wide?`	Generic dialog used by the settings pages
`AddRuleModal`	`onClose()`, `onCreated(rule: RiskRule)` (token comes from `useAuth`)	Builds a risk rule from `GET /settings/risk-rule-options` (no hand-written JSON), then `POST /settings/risk-rules` — or, on the Rule templates page, `POST /platform/risk-rule-templates`
`AddUserModal`	`onClose()`, `onCreated(user: AdminUser)`	`POST /settings/users` with company + role (or a platform admin); can generate a random 14-character initial password

Platform and tenancy components

Component	Props	Purpose
`PlatformShell` (`components/platform/`)	`active` tab, `title`, `description`, `actions?`, `children`	Page frame and tab bar of the platform-admin area: Companies, Users, Rule templates, Billing & usage, Processing queues
`CompanyPicker` (`components/`)	`note?`	Company selector for platform admins on company-scoped pages, with an amber read-only/audited note. Default note: "Read-only support access. Every view is recorded in the platform audit log (the company does not see it)."
`useActingCompany()` (`hooks/useActingCompany.tsx`)	—	`{isPlatformAdmin, companyId, platformCompanyParam, …}`: which company a platform admin is acting on (remembered per browser); company users always get their own. Pages pass `platformCompanyParam` as `company_id` to the API

Design system — `design-system/`

Small presentational pieces reused across pages: `Nav` (top navigation; props `active: NavItemId`, `onNewUploadClick?`; items filtered by `NAV_VISIBLE` — rank via `hasRank` for company users, explicit `is_platform_admin` checks for the Platform area; shows the company name next to the user), `PageHeader`, `StatCard` (KPI tile), `InfoChip`, `SeverityFinding` (`title`, `description`, `pointDelta`, `tone` — one triggered risk reason with its

weight), `AuditRow`. The static HTML mockups these were built against are in `stitch_docauth_document_review_platform/`.

UI primitives — `components/ui/`

`badge`, `button`, `card`, `input`, `label`, `progress`, `select` — shadcn/ui-style wrappers over Radix + Tailwind (class-variance-authority, `cn()` from `lib/utils`).

Routing and auth

- `ProtectedRoute` — token + resolved user required, else redirect to `/login`.
- `RoleRoute({minRole, platformAdmin, label})` (file `AdminRoute.tsx`) — renders `<Outlet/>` when the user's company role ranks at least `minRole` (`hasRank`), or when the user is a platform admin and `platformAdmin` is `"allow"` / `"only"` (`"only"` = platform admins exclusively); else an "access required" panel.
- `types/auth.ts` — `UserRole` (`user`, `reviewer_l1`, `reviewer_l2`, `platform_admin`), the rank table mirrored from the backend, `hasRank(role, minimum)` (always false for `platform_admin`, which is not on the ladder), `ROLE_LABELS` ("Reviewer L1" … "Platform Admin") and `ROLE_OPTIONS` (role pickers). Gate UI on `hasRank` / `is_platform_admin`, never on lists of role names.
- `hooks/useAuth.tsx` — `AuthProvider` / `useAuth()` exposing `token`, `user`, `isLoadingUser`, `login`, `logout`; token in `localStorage`; a token the backend rejects is cleared.

Data layer

- `api/client.ts` — `apiFetch<T>(path, {token, …})` (JSON, Bearer header, `ApiError(status, message, code?)`), `parseErrorDetail(body)` (reads `detail` whether it is a string or a `{code, message}` object) and `API_BASE_URL` (`VITE_API_BASE_URL` or `/api`). Uploads use `XMLHttpRequest` instead, purely for progress events.
- `api/auth.ts`, `api/cases.ts`, `api/settings.ts`, `api/platform.ts` — one function per endpoint.
- `lib/uploadLimits.ts` — `MAX_UPLOAD_BYTES` (10 MB, keep in step with the backend's `MAX_UPLOAD_SIZE_BYTES`), `ACCEPTED_UPLOAD_TYPES` (PDF), `clientUploadProblem(file)`.
- Every page except Login is a lazy route chunk (`App.tsx`).
- `types/` — hand-written TypeScript mirrors of the backend schemas (`types/case.ts` includes `SignatureMatchResult`, `getDocumentRole` — **kept in sync with the backend by hand**, so a new backend enum value needs a matching edit here, as the signature verdicts did).
- Forms use React Hook Form + Zod; the Zod schemas are hand-mirrored from the Pydantic models.